



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 7

DC MOTOR SPEED MEASUREMENT

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 7

Computer-Based Motor Speed Measurement

Objectives

1. To understand the principle and operation of computer-based measurement systems.
2. To learn to acquire and process signals automatically using a data-acquisition (DAQ) system.
3. To become familiar with signal conditioning for interfacing sensors with digital acquisition hardware.
4. To measure and analyse the rotational speed of a DC motor using pulse signals.
5. To develop skills in integrating hardware and software for real-time measurement and analysis.

Equipment / Software Required

- Personal computer (PC) running Windows, with a C/C++ development environment
- Data Acquisition (DAQ) card: Advantech PCI-1750 (16 isolated digital inputs, 16 isolated digital outputs, 1 isolated counter, 1 timer, 2500 VDC optical isolation), installed in the host PC, with driver and **Advantech Navigator.exe** installed
- Sensor trainer with DC motor and a 12-slot wheel-encoder pulse output
- Components for the student-designed filter and signal-conditioning circuit (op-amps, resistors, capacitors, Schmitt-trigger ICs, optocouplers)
- Digital storage oscilloscope (DSO), dual-channel, ≥ 50 MHz, with passive probes
- Connecting wires and breadboard

Introduction

In most measurement processes it is often necessary to acquire data automatically, without a human operator. A computer-based instrument has several advantages over conventional measuring instruments: it can acquire measurements quickly and accurately, sustain real-time control, and operate for extended periods with consistent precision.

A computer-based measurement system typically contains the following elements (Figure 7.1): a transducer, a signal-conditioning stage, data-acquisition hardware, and software running on the host PC.

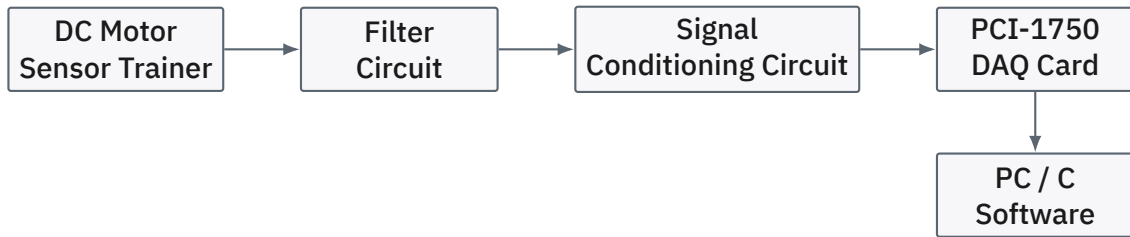


Figure 7.1: Computer-based measurement system: transducer, signal conditioning, DAQ hardware, and analysis software.

Safety & Hardware Protection

1. **Maximum voltage tolerance (0–5 V TTL limit).** The PCI-1750's digital input channels require standard 0–5 V TTL logic levels. Never connect a raw sensor signal exceeding 5 V directly to the DAQ card input. Measure the raw pulse voltage amplitude with the DSO *before* making any DAQ card connection.

Procedure

Activity 1 – Characterising the encoder signal and designing the filter

Objective: measure the encoder's pulse characteristics and design a signal-conditioning filter.

1. Power on the sensor trainer and verify motor rotation. Wire the encoder's raw output test point to Channel 1 of the oscilloscope.
2. Run the motor across its usable speed range and observe the encoder pulse train on the oscilloscope. Record the peak-to-peak voltage, rise time, overshoot, and any noise ripple on the raw signal.
3. At a representative operating speed, measure the pulse period and estimate the corresponding fundamental frequency from the oscilloscope trace.
4. From this measurement, determine a suitable cutoff frequency for the noise-rejection filter.
5. Design the necessary filter based on the cutoff frequency determined above. Justify your design choices.
6. Design the signal-conditioning circuit needed to present the filtered encoder pulses to the PCI-1750's digital input with valid 0–5 V TTL logic levels. Justify your design choices.

Activity 2 – Interfacing to the DAQ hardware

Objective: connect the conditioned encoder signal to the PC via the PCI-1750, and verify the connection before writing any software.

1. With the motor trainer powered off, connect the output of your signal-conditioning circuit to the designated digital input channel (e.g. DIO / Counter In) on the PCI-1750 terminal board. Connect a common ground between your breadboard and the PCI-1750 terminal board ground.
2. On Channel 2 of the oscilloscope, verify that the conditioned signal is a clean 0–5 V square wave with sharp edges and no false-trigger spikes, while keeping Channel 1 on the raw input for comparison. Adjust your filter's RC components if ringing occurs.
3. Launch **Advantech Navigator.exe** on the PC to verify hardware channel detection and pulse-counting activity — use this strictly as a hardware diagnostic to confirm the expected pulses are being registered, before writing any code.
4. Close Navigator.exe and launch your C development environment.



Figure 7.2: Signal path from the DC motor trainer to the host PC.

Activity 3 – Speed measurement software

Objective: implement software to compute and display motor speed (frequency and RPM) in real time, using two selectable measurement methods.

Two complementary methods can be used to convert the pulse train into a frequency and rotational speed, and your software should let the user select between them:

Method 1 – Fixed-Time (by counts, $1 \leq N \leq 99$): the user specifies a target pulse count N ; the software measures the elapsed time T_N for N pulses to occur, then computes

$$f = \frac{N}{T_N \times P} \text{ [rev/s]}, \quad \text{RPM} = 60f, \quad \omega = 2\pi f \text{ [rad/s]}$$

where P is the encoder's pulse resolution (pulses per revolution).

Method 2 – Fixed-Count (by period, $1 \leq T \leq 9 \text{ s}$): the user specifies a measurement window T seconds; the software counts the number of pulses N_T that occur within that window, then computes

$$f = \frac{N_T}{T \times P} \text{ [rev/s]}, \quad \text{RPM} = 60f, \quad \omega = 2\pi f \text{ [rad/s]}$$

In both methods, t_{start} and t_{stop} (marking the beginning and end of the measurement) are obtained from the PCI-1750's counter/timer using

$$\text{Elapsed time} = \frac{t_{\text{stop}} - t_{\text{start}}}{\text{CLK_TCK}}$$

where CLK_TCK is the counter's clock tick rate.

1. In C, write a program that:
 - initialises the PCI-1750 via the vendor driver/SDK (counter and digital-input registers).
 - lets the user select the measurement method (Fixed-Time, $1 \leq N \leq 99$; or Fixed-Count, $1 \leq T \leq 9$ s) and enter the corresponding parameter.
 - for the *Fixed-Time* method: times how long it takes for N encoder pulses to occur.
 - for the *Fixed-Count* method: counts how many encoder pulses occur within the window T .
 - computes the frequency, RPM, and angular speed ω using the relations above, updating at least every 500 ms.
 - displays the running value (frequency and RPM, with the active method indicated) in a simple HMI, updating in real time, with out-of-range error detection.
2. Interface your filter/conditioning circuit and the PCI-1750 together and perform measurements across the motor's input voltage range (e.g. 3.0 V to 12.0 V in 1.5 V increments), using both methods at each operating point. Compare your computed speed against a direct oscilloscope measurement of the encoder period at the same operating point.
3. Tabulate your results (Tables 7.2–7.3).
4. Plot computed speed (from both software methods) against the oscilloscope-measured speed, and plot relative error (%) against motor speed for both methods.
5. Discuss and comment on your results, including any discrepancy between the two software methods and the oscilloscope reference, and how each method's resolution changes with motor speed (see the trade-off noted above).

Table 7.1: By-counts method: frequency comparison table.

N	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Table 7.2: By-counts method — RPM comparison table.

N	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Table 7.3: By-period method: frequency comparison table.

T (s)	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Table 7.4: By-period method: RPM comparison table.

T (s)	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Report

1. Present your oscilloscope measurements of the raw and conditioned encoder pulse, and your resulting filter design (topology, calculations, and component values), together with the measured response of the built filter.
2. Present your design of the signal-conditioning stage between the filter and the PCI-1750 digital input, with justification, including the isolation and grounding measures you applied (see Safety & Hardware Protection above).
3. Compare your software-computed speed (both Fixed-Time and Fixed-Count methods) with the oscilloscope-measured speed across your tested operating points, and discuss the sources of any discrepancy.
4. Compare the Fixed-Time and Fixed-Count methods directly: at what motor speeds does each method give better resolution or a more stable reading, and why? Which would you recommend for a low-speed application, and which for a high-speed one?

5. What is the theoretical maximum measurable speed of this system, given the encoder resolution and your software's timing resolution?
6. Identify the primary sources of measurement error in your system (e.g. pulse jitter, counter quantisation error, sampling clock drift, motor speed ripple), and discuss their impact on overall RPM accuracy.

Deliverables

1. **C source code file (.c):** fully commented C software implementing pulse counting, timing calculations, RPM conversion, and HMI display.
2. **Technical design report (.pdf):** formal lab report detailing circuit schematics, component calculations, signal waveform captures, data tables, and error analysis.
3. **System demonstration:** a live bench demonstration showing real-time motor speed tracking on your C software HMI, alongside DSO verification.

References

- Advantech PCI-1750/PCI-1750SO user manual.

